

软件工程基础

软件工程与计算 II · 模块一 · 单元 1

当一段"正确"的代码让价值 5 亿美元的火箭在 37 秒后化为灰烬

课程	软件工程与计算 II
专业	软件工程
讲师	刘钦

今日学习路径

① 真实案例

一枚火箭的工程
失败如何发生

② 软件是什么

程序 $\neq$ 软件
三要素·七大特征
类型·现代形态

③ 软件工程

为什么需要"工程"
定义·规律·知识体系

④ 总结

知识框架
作业·预告



三个核心问题，今天结束时你应该能回答：

1. 软件与程序有什么区别？**软件的三要素是什么？**
2. 软件工程的定义是什么？**为何被称为"工程"？**
3. 现代软件工程知识体系（SWEBOK V4）**新增了哪些方向？**

PART 1 · 从一次价值 5 亿美元的失败开始

Ariane 5 · 37 秒后爆炸 · 5 亿美元消失

时间	事件
1996-06-04	阿丽亚娜 5 型火箭首飞，发射后 37 秒解体自毁
调查结论	64 位浮点数转换为 16 位整数时发生溢出，惯性参考系统（SRI）崩溃
根本原因	直接复用了 Ariane 4 的 SRI 软件模块，未重新验证在新环境中的适用性
代价	5 亿美元的火箭与卫星载荷，以及数年的研发延期

这是一次"代码完全正确，工程完全失败"的典型案列

SRI 模块通过了所有测试——但没有人验证它是否适合 Ariane 5 的新飞行轨迹

一个复用决策，如何传导为灾难？

非技术约束（进度与成本压力）

"Ariane 4 的 SRI 模块已验证，直接复用节省时间"

↓ 隐性假设 > 工程验证

数值范围假设未更新 ← 隐式约束缺失

集成测试不含新飞行场景 ← 验证遗漏

单点故障无容错设计 ← 系统约束缺失

↓ 软件"忠实"执行了错误的假设

SRI 溢出宕机 → 控制系统接收无效数据 → 触发自毁指令

↓ 37 秒

Ariane 5 首飞失败 = 5 亿美元 + 4 颗卫星

需求工程术语：此处的溢出源于 **隐式约束（Implicit Constraint）** 未被显式记录

停下来想一想

如果你是 Ariane 5 项目的软件工程师，
你会在哪个环节设置检查点？

和旁边的同学交流 1 分钟，然后分享你的想法

参考思路：

- 复用前是否做了**环境差异分析**（新火箭 vs 旧火箭的数值范围差异）？
- 数值约束是否被**显式记录**为软件规格的一部分？
- 集成测试是否覆盖了**新飞行剖面**下的极端数值？
- SRI 崩溃时，备份系统的行为是否有**明确规格**？

这个案例告诉我们什么？

核心问题	案例映射
软件工程为何不只是写代码？	代码正确，但验证、集成、变更管理全部缺失
隐式约束有多危险？	一个未被记录的数值范围假设，毁掉了整枚火箭
软件文档为什么重要？	约束没有进入文档 → 复用时无法被发现

💡 **关键洞察：** Ariane 5 事故的根因不是"程序员写错了代码"而是整个**软件工程过程**在关键环节系统性失效
(Lions et al., Ariane 501 Inquiry Board Report, 1996)

PART 2 · 软件是什么？

从信息到软件：一条变革之路

Information —▶ Computing —▶ Digital Computer —▶ Software —▶ AI ?

信息处理的三种形态：

形态	历史演变
信息的记录	泥板、莎草纸、龟壳、石头、竹子、纸、磁带、磁盘、光盘、硬盘
信息的交流	口头相传、信号、手语、电报、电话、卫星、互联网
信息的存储	书籍、图书分类、数据库、数据仓库、数据湖

软件的本质是**信息处理**——从泥板到云端，变的是介质，不变的是对信息的记录、交流与存储

数字计算的奠基（1930s）

硬件先驱：

- 微分计算器 — Vannevar Bush
- 继电器式计算机 — Konrad Zuse

理论先驱：

- 《论可计算数及其在判定问题上的应用》 — Alan Turing, 1936
- 《电子继电器可以实现布尔符号逻辑》 — Claude Elwood Shannon, 1937

💡 理论与硬件的双线并进，为软件的诞生奠定了基础

软件是硬件的一部分（1940s）

计算机	年代	备注
ABC 原型计算机	1939–1942	Atanasoff & Berry
Harvard Mark 1	1939–1944	IBM 实验室
ENIAC	1943–1946	世界首台通用电子数字计算机
EDVAC	1944–1949	存储程序概念，冯·诺依曼架构

早期编程的三个步骤：

1. 将问题**映射**到机器上（通常需要数周）
2. 通过操作开关和电缆将程序"装入"机器（又需数天）
3. **验证与调试**，通过单步执行来检查

EDVAC 引入了 **stored-program concept**（存储程序概念），奠定了冯·诺依曼架构——程序与数据存储在同一存储器中

软件工程 ≈ 硬件工程 (1950s)

商业计算机出现：

- Ferranti Mark I · UNIVAC I · LEO I · IBM 701 · IBM 650

高级编程语言诞生：

年份	语言	意义
1955	FORTRAN	首个高级语言，面向科学计算
1958	LISP	函数式编程鼻祖，AI 领域根基
1959	COBOL	面向商业应用，至今仍在银行系统运行

💡 1958 年，**John W. Tukey** 在《American Mathematical Monthly》上首次定义了运行在电子计算器上的程序——"Software"一词由此诞生

软件 ≠ 硬件 (1960s)

60 年代标志性事件：

- ASCII 美国信息交换标准码 · ATM · 信用卡
- IBM 信息管理系统 IMS 应用于阿波罗航天器
- 软件咨询业务兴起 · IBM S/360 · DEC PDP-1 小型机

软件与硬件的本质差异开始被认识到：

- 软件与现实世界关系更加密切，对需求的规格化更加困难
- 软件比硬件容易修改的多，并且不需要昂贵的生产线复制产品
- 软件没有损耗
- 软件不可见

Program = Algorithm + Data Structure (1970s ~ 1980s)

Niklaus Wirth (1976) 提出：算法与数据结构天然关联

例如，如果有一个有序列表，就应该使用针对有序列表最优的搜索算法

但"程序"还不是"软件"——这个认知即将改变

软件开发远比编程复杂 (1990s ~ 2010s)

应用爆发:

- 1990–1999 · 万维网的发展
- 2000–2009 · 社交网络 · 移动电商

软件工作量随规模变化 ([Jones 1996]):

规模 (功能点)	规模 (KLOC)	编码 (%)	文档编写 (%)	缺陷清除 (%)	管理和支持 (%)
1	0.1	70	5	15	10
10	1	65	7	17	11
100	10	54	15	20	11
1,000	100	30	26	30	14
10,000	1,000	18	31	35	16

💡 规模越大，编码占比越小——大型系统中编码仅占 18%，缺陷清除与文档编写远超编码工作量

大型软件系统的成本因素排序

规模为 100 万 LOC 的大型系统 ([Jones 2007]):

排名	工作内容
1	缺陷清除 (审查、测试、发现并修复 Bug)
2	纸质文档编写 (计划、规格说明、用户手册)
3	会议和交流 (顾客、团队成员、经理)
4	编程或编码
5	项目管理
6	变更控制

💡 **编码排第四!** ——缺陷清除、文档、沟通才是大型项目的主要成本

软件 ≠ 程序



要素	内容	Ariane 5 中缺失了什么?
程序	可执行的指令序列	SRI 代码本身是正确的 ✓
数据	程序处理与存储的信息	飞行参数范围未更新 ✗
文档	开发、使用、维护的说明	数值范围约束未记入文档 ✗
知识	隐性经验、最佳实践、领域理解	领域知识未被显式化和传承 ✗

IEEE 定义 (610.12-1990): 软件是计算机程序、规程及相关文档和运行计算机系统所需数据的**完整集合**

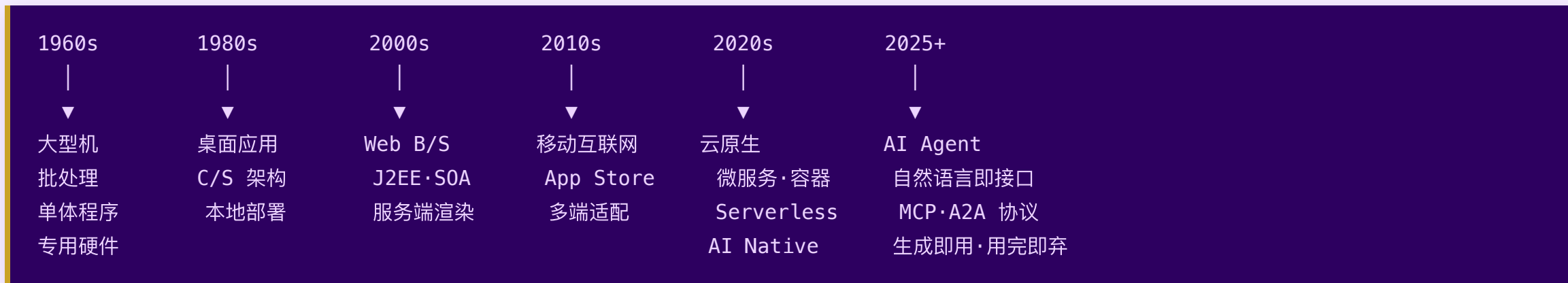
现代扩展定义: 软件 = 程序 + 数据 + 文档 + 知识

软件与硬件：七大本质差异

特征	说明	工程含义
无形性	看不见、摸不着	进度与质量难以直接观测
复杂性	逻辑交织，难以全面理解	需要抽象、分层、模块化
可复制性	复制成本趋近于零	分发成本低，但侵权风险高
易修改性	改动方便，但影响难预测	变更管理至关重要
退化性	不磨损，但随环境变化"腐化"	技术债·维护成本持续增长
依赖性	依赖运行环境（OS、库、网络）	环境一致性是工程问题
智力密集	本质是人的智力产品	人才是最核心的生产要素

💡 Ariane 5 的**依赖性**体现：SRI 模块依赖"Ariane 4 飞行参数范围"这一隐性环境假设

软件形态的六十年演变



传统软件的三种分类

类型	定义	典型例子	典型收费模式
系统软件	管理硬件资源，为其他软件提供平台	Linux、Windows、设备驱动	买断制 / 开源免费 / 随硬件捆绑
编程软件	辅助软件开发的工具集	VS Code、GCC、Git	开源免费 / 订阅制（JetBrains · GitHub Copilot）
应用软件	面向最终用户解决特定问题	微信、淘宝、银行核心系统	买断 / 订阅 / 免费+广告 / Freemium（基础免费·高级付费）

云时代的软件

云原生四要素（CNCF 定义）

微服务	容器化	DevOps	持续交付
独立部署的小型服务单元	Docker 封装运行环境	开发与运维一体化	自动化快速发布流水线

云时代的软件交付层次：IaaS · PaaS · SaaS

层次	全称	你负责	厂商负责	典型产品	典型收费模式
IaaS	基础设施即服务	OS · 运行时 · 应用代码	硬件 · 网络 · 虚拟化	AWS EC2 · 阿里云 ECS	按量计费（CPU · 存储 · 流量 · 时长）
PaaS	平台即服务	应用代码 · 数据	OS · 中间件 · 运行时	Vercel · Heroku · 腾讯云函数	按 API 调用次数 / 构建时长
SaaS	软件即服务	使用与配置	全部底层	钉钉 · Salesforce · 腾讯文档	订阅制（按用户数 · 月付 · 年付）

💡 **Serverless = FaaS + BaaS**：开发者只需关注业务函数，无需管理服务器

软件的现代应用领域

领域	代表技术与产品
云	Docker · Kubernetes · Istio
众	众筹 · 众包
大数据	Spark · Hadoop · Flink · Kafka · Elasticsearch · ClickHouse
AI	AlphaGo · AlphaCode · ChatGPT · Sora
短视频 · Web3.0	新一代内容平台与去中心化应用

LLM 时代：软件形态的第五次跃迁

四种 LLM 软件交互模式的演进

① 问答模式 ChatGPT 初期	② 问数模式 Text-to-SQL · BI	③ 助手模式 Copilot · Cursor	④ 智能体模式 AutoGPT · Manus · OpenClaw
你问 → AI 答 单轮，无记忆	说人话 → 查数据 结构化输出	全程陪伴 · 辅助 上下文持续累积	委托任务 · 自主完成 规划 + 工具调用 + 执行

三种正在重写"什么叫软件"的新范式

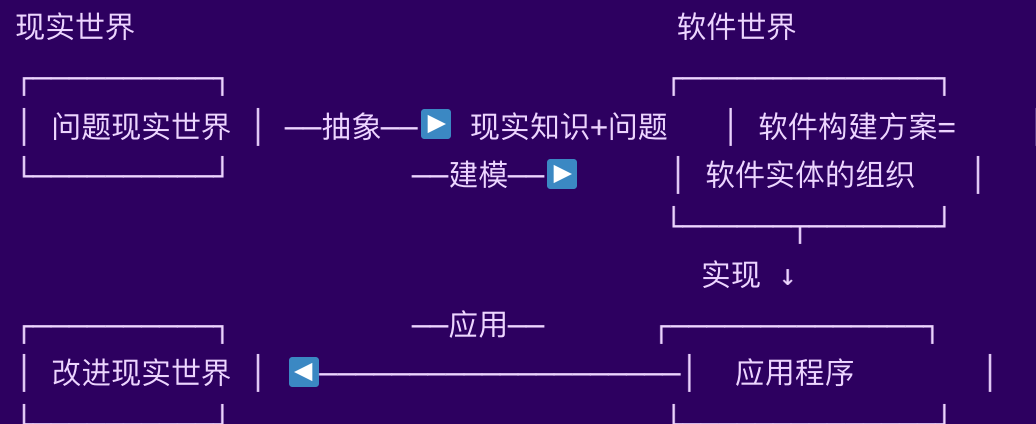
新范式	含义	对比传统模式
Agent as Service	服务单元从 API 接口变为"会思考的自主代理"	REST 接口调用 → 委托意图给 Agent
Context as Service	记忆与上下文跨会话持久化，成为产品核心资产	无状态请求 → 有记忆的长期协作者
抛弃式交互	生成即用、用完即弃，一次性脚本/工件无需长期维护	代码持续维护 → 按需生成、随时重生成

💬 **引发思考：**当软件可以用自然语言"生成"而非"编写"——需求分析、测试、部署的边界将如何改变？软件工程师的不可替代价值在哪里？

应用软件基于现实又高于现实

软件不是凭空创造的产物——它扎根于现实，又反过来重塑现实。

- **来源于现实** — 应用软件被开发的目的和意图来源于现实世界的问题。
- **基于现实** — 应用软件必须基于现实才能解决问题。
- **改进现实** — 软件最终要被用于现实并改进现实。



What is software? — Summary

- Software is **independent of hardware**
- Software is a **tool**
- Software = **programs + documents + data + knowledge**
- Software development is **much more complicated** than programming
- Application software **originate from the reality**, and reversely **improve the reality**

PART 3 · 什么是软件工程？

软件危机：为什么需要"工程"?

年代	标志性事件
1968	NATO 软件工程会议, "软件危机"概念正式提出
1970s	瀑布模型出现, 引入工程化流程管理软件开发
1980s	结构化方法、CMMI 成熟度模型
1990s	面向对象、统一过程 (RUP)
2000s	敏捷宣言, Scrum/XP, 响应快速变化
2010s	DevOps · 云原生 · 全球分布式协作
2020s	AI for SE · SE for AI——新一轮范式转变

💡 Barry Boehm (2006): 软件工程的每一次革命, 都是对前一阶段**未解决问题**的系统性回应

软件工程的定义

IEEE 标准定义 (610.12-1990)

- (1) The application of a **systematic, disciplined, quantifiable** approach to the **development, operation, and maintenance** of software; that is, the application of engineering to software.
- (2) The study of approaches as in (1).

软件工程是将系统化的、规范化的、可量化的方法
应用于软件的开发、运行和维护的过程；
即将工程化的方法应用于软件。

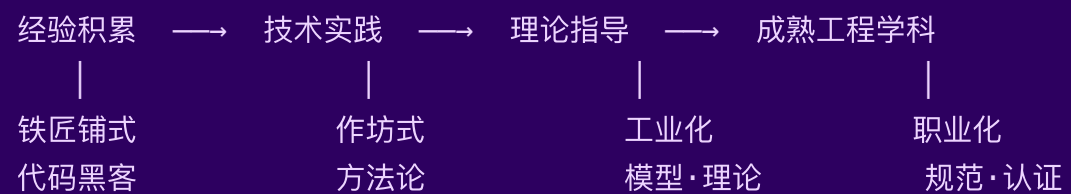
"工程"的定义 (ECPD / ABET)

The **creative application** of scientific principles to design or develop structures, machines, apparatus, or manufacturing processes, or works utilizing them singly or in combination; or to construct or operate the same with full cognizance of their design; or to forecast their behavior under specific operating conditions; all as respects **an intended function, economics** of operation or **safety** to life and property.

CCSE：对工程师的理解

- 工程师通过一系列的讨论决策，仔细评估项目的可选活动，并在每个决策点选择一种在当前环境中适合当前任务的方法进行工作
- 工程师需要对某些对象进行**度量**，有时需要定量的工作
- 软件工程师强调项目设计过程的**纪律性**，这是团队高效工作的条件
- 工程师可胜任研究、开发、设计、生产、测试、构造、操作、管理，以及销售、咨询和培训等**多种角色**
- 工程师们需要在某些过程中使用**工具**，选择和使用合适的工具是工程的关键要素
- 工程师们能够**重用设计和设计制品**

工程学科的发展规律 (M. Shaw, 1990)



软件工程目前处于**理论指导** → **成熟工程**的过渡阶段

M. Shaw：软件工程学科的五大特征

特征	说明
解决实际问题	工程学解决实际问题，而这些问题来源于工程领域之外的人——消费者
应用科学知识指导	工程学不依赖于个人技能，而是以科学知识为指导，按照特定方法与技术进行规律性的设计、分析等活动
成本效益比有效	工程学不单单只是解决问题，它要有效利用所有资源，至少成本要低于效益
构建机器或事物	工程学强调构建实物工具，例如机器、事物等，并利用实物工具来解决问题
以服务人类为最终目的	工程学考虑的不是单个客户的需要，而是要运用技术和经验实现全社会的进步

工程的四大要素

要素	在软件工程中的角色
Problem（问题）	动机（Motivation）——现实世界的混乱状态驱动工程
Scientific Knowledge（科学知识）	工具（Instrument）——计算科学作为科学基础
Solution/Machine（解决方案）	目标（Objective）——虚拟计算机 = 通用机器 + 特定解决方案
Cost-Benefit Effective（成本效益有效）	条件（Condition）——追求足够好，而不是最好

对软件工程的理解

- 软件工程是一种**工程活动**
- 软件工程的动机是**解决实际问题**
- 软件工程是**科学性、实践性和工艺性**并重的
- 软件工程追求**足够好**，不是最好
- 软件工程真正的产品是基于虚拟计算机的**软件方案**
- 软件工程的最终目的是要**促进整个社会的进步**

Engineering = Science + Principle + Art

SWEBOK V4 (2024): 软件工程知识体系的重大更新

知识域	状态	意义
软件需求 / 设计 / 构造 / 测试 / 维护	核心保留	基础不变
软件配置管理 / 工程管理 / 过程 / 质量	核心保留	基础不变
软件架构	V4 新增	架构从设计中独立，地位提升
软件工程运维	V4 新增	DevOps 时代运维纳入 SE 范畴
软件安全	V4 新增	安全从附录升级为独立知识域
AI for SE	V4 新增	AI 辅助软件工程全过程
SE for AI	V4 新增	AI 系统的工程化方法

💡 SWEBOK V4 的更新，是对 2020s 软件行业核心挑战的直接回应

SWEBOK V3 → V4 对照表

SWEBOK V3	SWEBOK V4	状态
Introduction	Introduction	Open
1. Software Requirements	1. Software Requirements	closed
	2. Software Architecture	V4 新增
2. Software Design	3. Software Design	closed
3. Software Construction	4. Software Construction	closed
4. Software Testing	5. Software Testing	closed
	6. Software Engineering Operations	V4 新增
5. Software Maintenance	7. Software Maintenance	Open
6. Software Configuration Management	8. Software Configuration Management	Open
7. Software Engineering Management	9. Software Engineering Management	closed
8. Software Engineering Process	10. Software Engineering Process	Open
9. Software Engineering Models and Methods	11. Software Engineering Models and Methods	Open
10. Software Quality	12. Software Quality	closed
	13. Software Security	V4 新增
11. SE Professional Practice	14. SE Professional Practice	closed

SWEBOK V3 知识域全景

技术知识域 (5 个核心域):

软件需求	软件设计	软件构造	软件测试	软件维护
└需求基础	└设计基础	└构造基础	└测试基础	└维护基础
└需求过程	└设计关键问题	└管理构造	└测试级别	└关键问题
└需求获取	└软件结构与	└实际考虑	└测试技术	└维护过程
└需求分析	└体系结构		└需求分析	└维护技术
└需求规格说明	└设计质量		└与测试相关	
└需求确认	└的分析与评价		└的度量	
└实际考虑	└设计符号		└测试过程	
	└设计策略与方法			

管理知识域 (6 个域 + 相关学科):

软件配置管理	软件工程管理	软件工程过程	软件工程工具	软件质量	相关学科知识域
└过程管理	└启动和	└过程实施	└需求工具	└质量基础	└计算机工程
└标识	└范围定义	└与改变	└设计工具	└质量过程	└计算机科学
└控制	└项目计划	└过程定义	└构造工具	└实际考虑	└管理
└状态簿记	└项目实施	└过程评定	└测试工具		└数学
└审计	└审评与评价	└过程和	└维护工具		└项目管理
└发布管理	└关闭	└产品度量	└配置管理工具		└质量管理
和交付	└度量		└过程工具		└软件人类
			└质量工具		└工程学
					└系统工程

软件开发：活动 · 产物 · 角色分工



角色	核心职责
产品经理	定义"做什么", 管理需求优先级, 对齐业务目标
架构师	定义"怎么做", 设计技术方案, 把控技术风险
开发工程师	实现功能, 编写可维护的代码
测试工程师	验证质量, 发现并跟踪缺陷
运维工程师	保障系统稳定运行, 管理发布流程

软件开发活动与产物（详细版）

活动	目标	产物
Software Requirement	What（做什么）	SRS（需求规格说明书）
Software Design	How（怎么做）	SDD（设计文档）
Software Construction	Build（构建）	代码与可执行文件
Software Testing	Are you building the "right" thing? Are you building it "right"?	测试报告
Software Deliver	Install（安装部署）	用户文档与系统文档
Software Maintenance	Revolution（演化）	新版本软件

角色分工（详细版）

角色	职责描述
需求工程师	又称需求分析师。承担需求开发任务，通常由多人组成需求工程师小组，在首席需求工程师的领导下开展工作
软件体系结构师	承担软件体系结构设计任务，通常也是由多人组成小组，通常一个团队只有一个软件体系结构师小组
软件设计师	承担详细设计任务。在体系结构设计完成之后，将部件分配给不同的开发小组
程序员	承担软件构造任务。程序员与软件设计师通常是同一批人
人机交互设计师	承担人机交互设计任务，可以与软件设计师是同一批人，也可以是独立人员
软件测试人员	承担软件测试任务，通常需要 独立于其他的开发人员角色
项目管理人员	负责计划、组织、领导、协调和控制软件开发的各项工作。更多得是 协调者 而非监控者
软件配置管理人员	管理软件开发中产生的各种制品，对重要制品进行标识、变更控制、状态报告等
质量保障人员	在生产过程中监督和控制软件产品质量
培训和支持人员	负责软件移交与维护任务
文档编写人员	专门负责写作软件开发各种文档，让宝贵的人力资源从繁杂的文档化工作中解放出来

LLM 时代，软件工程如何演变？

活动 / 角色	关键变化	新增技能要求
需求工程	AI 辅助用户访谈、自动生成用户故事与验收标准	辨别 AI 提炼是否失真，能精准提问
系统设计	AI 生成多套方案供选择，人负责决策与取舍	RAG · Agent · MCP 等 AI 系统架构能力
编码实现	Copilot 补全为主，重心转向 代码审查与集成	Prompt 工程 · 识别代码幻觉与安全漏洞
测试验证	AI 批量生成测试用例，覆盖率大幅提升	专项测试 AI 输出：幻觉 · 边界 · 偏见
运维部署	新增 LLM 可观测性：推理延迟、Token 成本	LLMOps：模型版本管理 · 输出漂移监控
角色边界	产品 / 开发 / 测试界限模糊，全栈 AI 工程师涌现	T 型能力 + AI 协作素养

💡 **不变的内核：**理解用户真实需求 · 管理系统复杂度 · 保障质量与可靠性 · 工程伦理——AI 改变的是**效率与手段**，对正确性与用户价值的责任**只增不减**

PART 4 · 总结与课后任务

今天建立的知识框架

软件工程基础

- 软件
 - 定义：程序 + 数据 + 文档 (IEEE 610.12-1990)
 - 七大特征：无形·复杂·可复制·易修改·退化·依赖·智力密集
 - 现代形态：云原生 · 微服务 · Serverless · AI Native
- 软件工程
 - 定义：系统化 · 规范化 · 可量化 (IEEE 610.12-1990)
 - 发展规律：经验 → 实践 → 理论 → 成熟 (M. Shaw)
 - 知识体系：SWEBOK V4 (新增 AI for SE / SE for AI)
 - 开发角色：产品经理 · 架构师 · 开发 · 测试 · 运维

从 Ariane 5 到你的课程项目：工程方法的每一项，都是真实失败换来的教训

今天最重要的三个结论

1. 软件 ≠ 程序

程序是软件的一部分；文档与数据同样是软件的核心组成
缺少文档的软件，是下一次 Ariane 5 事故的隐患

2. 工程方法来自失败的积累

每一条软件工程规范，背后都有一次代价高昂的教训
SWEBOK 的每次更新，都在回应行业还未解决的问题

3. AI 改变工具，不改变工程本质

AI for SE 让工具更智能；但系统化·规范化·可量化的工程原则不变
SWEBOK V4 同时新增了 AI for SE 和 SE for AI——二者缺一不可

课后作业（提交至 SEECODER-SE · 基础模块）

个人独立完成，截止：下次课前 48 小时

#	任务	要求
1	软件三要素分析	选取一款你常用的 App，找出它的程序、数据、文档各对应什么，100 字说明
2	软件类型辨析	列举手机上 5 款 App，按三种类型分类，并说明分类理由
3	SWEBOK V4 探索	选择一个 新增知识域 ，200 字说明它解决了什么现实问题
4	反思文字	150 字：如果 Ariane 5 团队完整记录了数值范围约束（文档要素），哪个环节可以被阻止？

 **提示：**第 4 题没有标准答案，重点在于你的**工程推理过程**

下节课：模块一 · 单元 2

项目管理基础

- 软件项目的特殊挑战：为什么软件项目经常延期？
- 三种团队结构：主程序员 · 民主 · 开放——哪种更适合你的项目？
- 软件质量保障：评审 · 测试 · 审计的分工
- 软件配置管理：Git 分支策略与变更控制流程

课前思考：

你参与过的团队项目中，最常出现哪种协作问题？

（代码互相覆盖？职责不清？进度失控？）

这些问题背后有没有共同的工程原因？

本节课结束

"软件工程的每一条规范，都是某次灾难换来的教训。"
"写出正确的代码只是第一步——让正确的代码在正确的地方运行，才是工程。"

软件工程与计算 II · 模块一 · 单元 1